



Open Ended Lab 11 - Worksheet

Name: Hassaan ul Quadir , Maaz Ali , Abdul Rehman	ID: hu10322 , ma10339 , ar09828	Section: T4
---	---------------------------------------	-------------

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Design Module Codes for **Program Counter**, **PC Adder**, **Branch Adder**, **Mux2**, **immGen**.

```
`timescale 1ns / 1ps

module ProgramCounter (
    input wire clk,
    input wire reset,
    input wire [31:0] pc_in,
    output reg [31:0] pc_out
);

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            pc_out <= 32'b0;
        end else begin
            pc_out <= pc_in;
        end
    end

endmodule
```

ProgramCounter



```
`timescale 1ns / 1ps

module pcAdder (
    input wire reset,
    input wire [31:0] pc_out,
    output reg [31:0] next_pc
);

    always @(*) begin
        if (reset) begin
            next_pc = 32'b0;
        end else begin
            next_pc = pc_out + 32'd4;
        end
    end

endmodule
```

pcAdder

```
`timescale 1ns / 1ps

module branchAdder (
    input wire reset,
    input wire [31:0] pc_out,
    input wire [31:0] imm_ext,
    output reg [31:0] br_pc
);

    always @(*) begin
        if (reset) begin
            br_pc = 32'b0;
        end else begin
            br_pc = pc_out + (imm_ext << 1);
        end
    end

endmodule
```

branchAdder



```
`timescale 1ns / 1ps

module mux (
    input wire reset,
    input wire [31:0] d0,
    input wire [31:0] d1,
    input wire sel,
    output reg [31:0] out
);

    always @(*) begin
        if (reset) begin
            out = 32'b0;
        end else begin
            out = sel ? d1 : d0;
        end
    end

endmodule
```

mux2

```
`timescale 1ns / 1ps

`timescale 1ns / 1ps

module immGen (
    input wire reset,
    input wire [31:0] instruction,
    output reg [31:0] imm_ext
);

    wire [6:0] opcode = instruction[6:0];

    always @(*) begin
        if (reset) begin
            imm_ext = 32'b0;
        end else begin
            case (opcode)
                7'b0010011, 7'b0000011: begin
                    imm_ext = {{20{instruction[31]}}, instruction[31:20]};
                end
                7'b0100011: begin
                    imm_ext = {{20{instruction[31]}}, instruction[31:25], instruction[11:7]};
                end
                7'b1100011: begin
                    imm_ext = {{20{instruction[31]}}, instruction[7], instruction[30:25], instruction[11:8]};
                end
                default: begin
                    imm_ext = 32'b0;
                end
            endcase
        end
    end

endmodule
```

immGen



Testbench:

```
`timescale 1ns / 1ps

module tb_pc_increment();
    reg clk, reset;
    wire [31:0] pc_out, next_pc;

    // Instantiate PC and Adder
    ProgramCounter pc_inst (.clk(clk), .reset(reset), .pc_in(next_pc), .pc_out(pc_out));
    pcAdder adder_inst (.reset(reset), .pc_out(pc_out), .next_pc(next_pc));

    // Clock generation
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; #10;
        reset = 0;

        $display("Time | PC_Current | PC_After_Add4");
        repeat(4) begin
            @(posedge clk);
            #1; // Wait for logic to settle
            $display("%4t | %h | %h", $time, pc_out, next_pc);
        end
        $finish;
    end
endmodule
```

Test for pcAdder

```
`timescale 1ns / 1ps

module tb_branch_target();
    reg reset, pcsync_sel;
    reg [31:0] pc_current, imm_ext, pc_plus4;
    wire [31:0] pc_branch, pc_next;

    // Instantiate Branch Adder and the PC Mux
    branchAdder br_add (.reset(reset), .pc_out(pc_current), .imm_ext(imm_ext), .br_pc(pc_branch));
    mux pc_mux (.reset(reset), .d0(pc_plus4), .d1(pc_branch), .sel(pcsync_sel), .out(pc_next));

    initial begin
        reset = 0;
        pc_current = 32'h00000004; // Start at PC = 4
        pc_plus4 = 32'h00000008; // Normal next PC
        imm_ext = 32'hFFFFFFF; // Immediate is -1
        pcsync_sel = 1; // Force Branch Taken

        #10;
        $display("PC: %d | Imm: %d | Target (PC + Imm<<1): %d | PC_Next: %d",
            pc_current, $signed(imm_ext), $signed(pc_branch), $signed(pc_next));

        if (pc_next == 32'd2)
            $display("SUCCESS: Jumped to PC 2 (4 + (-1 * 2))");
        else
            $display("FAILURE: Expected 2, got %d", pc_next);

        $finish;
    end
endmodule
```

Test for branchAdder



```
`timescale 1ns / 1ps

module tb_imm_gen();
    reg reset;
    reg [31:0] instruction;
    wire [31:0] imm_ext;

    immGen uut (.reset(reset), .instruction(instruction), .imm_ext(imm_ext));

    initial begin
        reset = 0;

        // Test 1: Positive I-type (addi x1, x2, 10)
        instruction = 32'h00A10093;
        #10;
        $display("Inst: %h | Imm_Ext: %h | Expected: 0000000a", instruction, imm_ext);

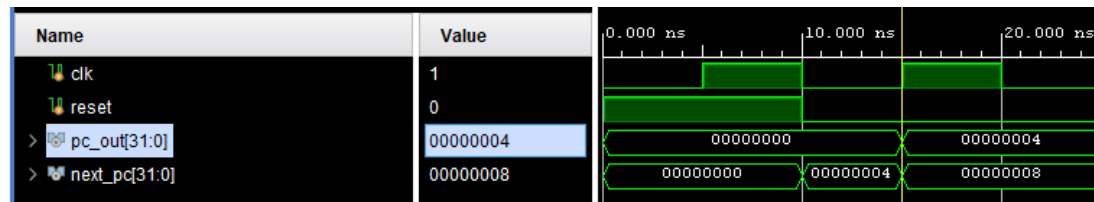
        // Test 2: Negative I-type (addi x1, x2, -1)
        instruction = 32'hFFF10093;
        #10;
        $display("Inst: %h | Imm_Ext: %h | Expected: ffffffff", instruction, imm_ext);

        // Test 3: Negative B-type (beq x0, x0, -4)
        instruction = 32'hFE000CE3;
        #10;
        $display("Inst: %h | Imm_Ext: %h | Expected: ffffffff", instruction, imm_ext);

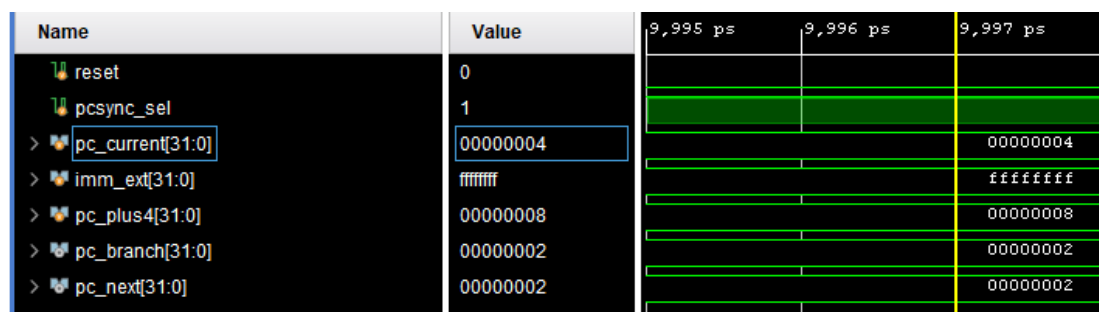
        $finish;
    end
endmodule
```

Test for immGen

Waveforms:



Simulation for pcAdder



Simulation for branchAdder



Name	Value	0.000 ns	5.000 ns
reset	0		
> instruction[31:0]	00a10093	00a10093	
> imm_ext[31:0]	0000000a	0000000a	

Simulation for immGen (Instruction: addi x1 , x2 , 10)

Task 2

Design module code of *TopLevelProcessor* module:

```
`timescale 1ns / 1ps

`timescale 1ns / 1ps

module TopLevelProcessor(
    input wire clk,
    input wire reset,
    input wire [15:0] switches_in,
    output wire [15:0] leds_out
);

    // --- Internal Wires ---
    wire [31:0] pc_current, pc_next, pc_plus4, pc_branch;
    wire [31:0] instruction;
    wire [31:0] imm_ext;
    wire [31:0] rd1, rd2, write_back_data;
    wire [31:0] alu_operand_b, alu_result;
    wire [31:0] mmio_read_data; // Data coming FROM the address decoder
    wire [3:0] alu_ctrl;
    wire [1:0] alu_op;
    wire reg_write, mem_read, mem_write, alu_src, mem_to_reg, branch, alu_zero;
    wire pcsync_sel;

    // --- 1. Instruction Fetch (IF) ---
    ProgramCounter PC_Inst (
        .clk(clk), .reset(reset), .pc_in(pc_next), .pc_out(pc_current)
    );

    pcAdder PC_Add4 (
        .reset(reset), .pc_out(pc_current), .next_pc(pc_plus4)
    );

    instructionMemory IM_Inst (
        .reset(reset), .instAddress(pc_current), .instruction(instruction)
    );

    // --- 2. Decode & Control (ID) ---
    MainControl Control (
        .opcode(instruction[6:0]), .RegWrite(reg_write), .ALUOp(alu_op),
        .MemRead(mem_read), .MemWrite(mem_write), .ALUSrc(alu_src),
        .MemtoReg(mem_to_reg), .Branch(branch)
    );

    ALUControlUnit ALU_Control (
        .ALUOp(alu_op), .funct3(instruction[14:12]), .funct7(instruction[31:25]),
        .ALUControl(alu_ctrl)
    );

endmodule
```



```
RegisterFile RegFile (
    .clk(clk), .rst(reset), .RegWrite(reg_write),
    .rs1(instruction[19:15]), .rs2(instruction[24:20]), .rd(instruction[11:7]),
    .wd(write_back_data), .rd1(rd1), .rd2(rd2)
);

immGen Imm_Gen_Inst (
    .reset(reset), .instruction(instruction), .imm_ext(imm_ext)
);

// --- 3. Execute (EX) ---
mux ALU_Mux (
    .reset(reset), .d0(rd2), .d1(imm_ext), .sel(alu_src), .out(alu_operand_b)
);

ALU ALU_Inst (
    .reset(reset), .A(rd1), .B(alu_operand_b), .ALUControl(alu_ctrl),
    .ALUResult(alu_result), .Zero(alu_zero)
);

// --- 4. Memory & I/O (MEM) ---
// The Address Decoder manages DataMemory, switches, and leds internally
addressDecoder MMIO_System (
    .clk(clk),
    .rst(reset),
    .address(alu_result),
    .readEnable(mem_read),
    .writeEnable(mem_write),
    .writeData(rd2),
    .switches_in(switches_in),
    .readData(mmio_read_data),
    .leds_out(leds_out)
);

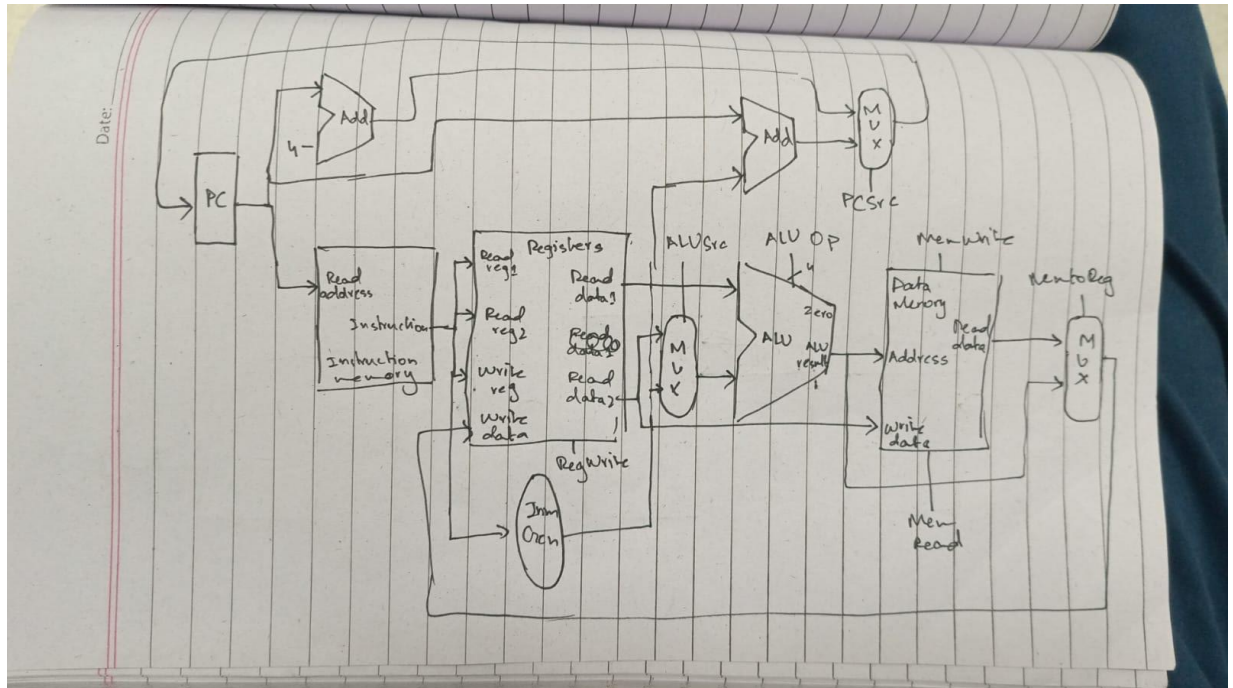
// --- 5. Write Back (WB) ---
mux WB_Mux (
    .reset(reset), .d0(alu_result), .d1(mmio_read_data), .sel(mem_to_reg), .out(write_back_data)
);

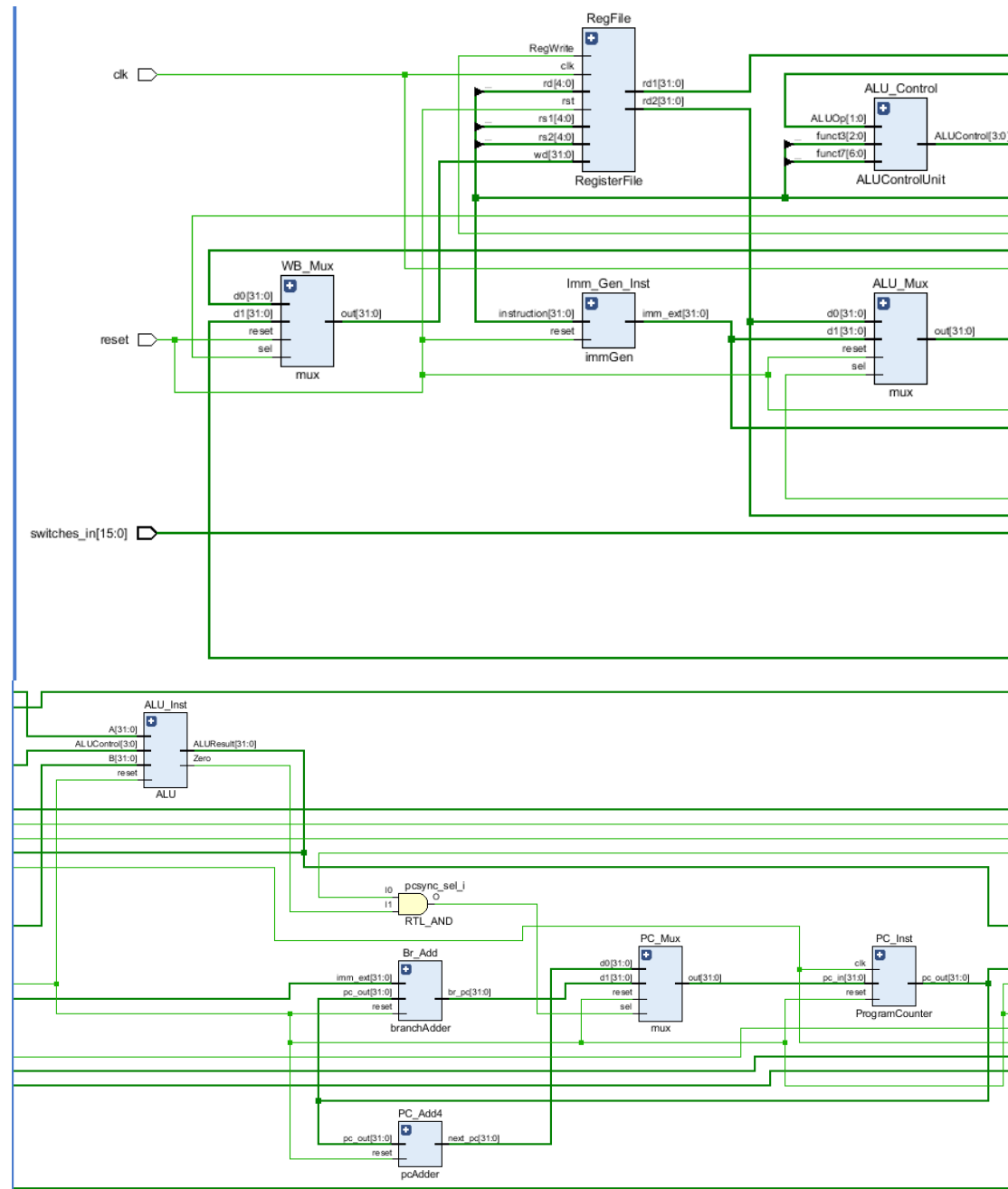
// --- Branching Logic ---
branchAdder Br_Add (
    .reset(reset), .pc_out(pc_current), .imm_ext(imm_ext), .br_pc(pc_branch)
);

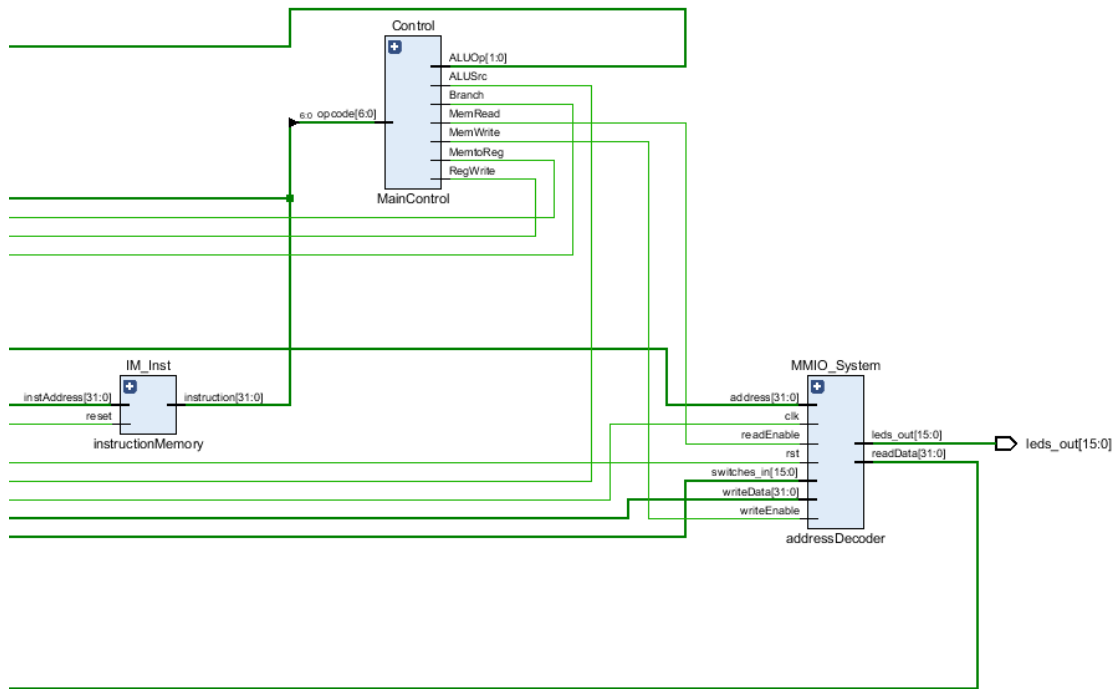
assign pcsync_sel = branch & alu_zero;

mux PC_Mux (
    .reset(reset), .d0(pc_plus4), .d1(pc_branch), .sel(pcsync_sel), .out(pc_next)
);

endmodule
```

*Datapath Diagram:*

*RTL Schematic:*



Utilization Report:

Design Runs		Utilization					Timing	
Q		≡	⬇	%	Hierarchy			
Name		^1	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Bonded IOB (106)	BUFGCTRL (32)
▼ N TopLevelProcessor			1100	1031	224	52	34	1
I ALU_Inst (ALU)			0	0	0	0	0	0
I Br_Add (branchAdder)			0	0	0	0	0	0
> I MMIO_System (addressDecoder)			0	32	0	0	0	0
I PC_Add4 (pcAdder)			0	0	0	0	0	0
I PC_Inst (ProgramCounter)			595	7	1	0	0	0
I RegFile (RegisterFile)			505	992	223	52	0	0

Timing Analysis Report:

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 7.880 ns	Worst Hold Slack (WHS): 0.283 ns	Worst Pulse Width Slack (WPWS): 9.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 2023	Total Number of Endpoints: 2023	Total Number of Endpoints: 1032
All user specified timing constraints are met.		



Lab 11: Design and FPGA Implementation of a Single-Cycle RISC-V Processor

Points Distribution

Task No.	OE2 (Design)	OE5 (Report)	OE6 (Reflection)	AR7
Task 1	/40	-	-	/20
Task 2	/20	/30		
Viva			/10	
Total Points	/80			/20
CLO Mapped	CLO 3			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.